Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Cycle de vie des processus, ordonnancement préemptif, filiation PID/PPID, appels système fork/exec et communication inter-processus par signaux POSIX.

Synthèse exécutive : Ce cours aborde les fondements majeurs de l'architecture des systèmes d'exploitation de type Unix/Linux (module R1.04). Il explicite la transition fondamentale entre un programme statique stocké sur le disque dur et un processus actif s'exécutant en mémoire RAM. Pour permettre l'exécution fluide et simultanée de multiples applications sur un même système, le noyau (kernel) met en œuvre le multitâche préemptif au moyen d'un ordonnanceur (scheduler) qui fait évoluer chaque processus à travers un cycle de vie strict à trois états (Élu, Prêt, Bloqué). Le cours décortique l'arborescence généalogique des processus (PID/PPID) issue du processus initial systemd/init (PID 1), le fonctionnement des appels système de création fork() et exec(), ainsi que le mécanisme universel de communication et de contrôle inter-processus : les signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour qu'un ordinateur exécute du code, le système d'exploitation doit faire basculer le programme (ensemble d'instructions statiques stockées sur le stockage secondaire) vers l'état de processus (entité active chargée en mémoire vive RAM). Le noyau Linux orchestre ce fonctionnement grâce au multitâche préemptif : le temps du processeur est découpé en minuscules quanta de temps attribués successivement aux processus éligibles. L'ordonnanceur (scheduler) gère le passage dynamique des processus entre trois états fondamentaux : 1. Élu (Running) : Le processus est pris en charge par un cœur du processeur et exécute ses instructions machine. 2. Prêt (Ready) : Le processus dispose de toutes les ressources mémoire nécessaires mais attend que l'ordonnanceur lui attribue son prochain quantum de temps CPU. 3. Bloqué (Waiting / Blocked) : Le processus a déclenché une requête nécessitant une attente externe (ex: lecture de fichier sur disque, attente d'un paquet réseau ou saisie utilisateur au clavier). Pour éviter de pénaliser les performances globales, le noyau libère le CPU et place le processus en attente d'événement. Dès que l'événement survient, le processus repasse obligatoirement par l'état 'Prêt' avant de pouvoir être réélu. Tous les processus au sein du système d'exploitation s'inscrivent dans une arborescence généalogique stricte. Lors de l'initialisation du système, le noyau crée le tout premier processus utilisateur, nommé systemd (ou init sur les distributions plus anciennes), auquel est assigné le PID 1. Lors de la création d'un sous-processus (par exemple lorsqu'un utilisateur lance une commande dans un terminal Bash), le terminal (processus parent) appelle la fonction système fork() pour se cloner. Le processus fils ainsi généré possède un nouveau PID et hérite du PID du parent (PPID). L'enfant exécute ensuite l'appel système exec() pour substituer sa carte mémoire par le code binaire du programme souhaité.

- • Programme vs Processus : Un programme est un fichier binaire passif résidant sur le disque dur ; un processus est une instance dynamique active chargée en mémoire RAM, consommant des ressources CPU et possédant un contexte d'exécution.
- • Multitâche préemptif & Ordonnanceur : Le noyau alloue de très courts quanta de temps processeur (tranches de temps) aux processus éligibles pour simuler un parallélisme d'exécution sur le système.
- • PCB (Process Control Block) : Structure de données interne au noyau regroupant toutes les métadonnées d'un processus : état du processeur, carte mémoire (stack, heap, code, data), identifiants (PID, PPID, UID), priorité et descripteurs de fichiers ouverts.
- • Arborescence PID/PPID & Processus Racine : Filiation généalogique stricte où chaque processus possède un identifiant unique (PID) et dérive d'un processus père (PPID). Le processus racine utilisateur est systemd ou init (PID 1).
- • Appels système fork() et exec() : fork() duplique le processus parent à l'identique pour générer un processus enfant ; exec() remplace la mémoire du processus enfant par le code d'un nouveau programme binaire à exécuter.

• Cycle de Vie à 3 États : Élu (Running - occupe le CPU et exécute son code), Prêt (Ready - dispose de ses ressources RAM et attend du temps CPU), Bloqué (Waiting/Blocked - suspendu en attente d'une opération d'E/S disque, réseau ou frappe clavier).

Cas concret enseignant : *L'analogie du livre de recettes : Le professeur compare le programme sur disque dur à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, consommant de l'espace sur le plan de travail en RAM, du temps de préparation CPU et manipulant des ustensiles ou descripteurs de fichiers).*

Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX représentent la forme de communication inter-processus (IPC - Inter-Process Communication) la plus rudimentaire et universelle des systèmes Unix. Un signal est une notification asynchrone : lorsqu'il est délivré à un processus cible, l'exécution courante de ce dernier est interrompue instantanément pour traiter l'événement. À la réception d'un signal, trois options s'offrent au processus selon sa programmation : 1. Exécuter l'action par défaut définie par le système (généralement la terminaison du processus ou son ignorance). 2. Ignorer le signal (ignore). 3. Intercepter le signal (catch) en déroutant le fil d'exécution vers une fonction dédiée appelée 'gestionnaire de signal' (signal handler). Cela permet par exemple à un logiciel de traitement de texte d'intercepter SIGINT ou SIGTERM pour afficher le message : 'Voulez-vous sauvegarder vos modifications avant de quitter ?'. Cependant, pour conserver un contrôle absolu sur le système et permettre à l'administrateur de neutraliser n'importe quel processus défaillant ou malveillant, la norme POSIX définit deux exceptions majeures : SIGKILL (Signal 9) et SIGSTOP (Signal 19). Ces deux signaux sont interceptés et exécutés directement par le noyau lui-même. Aucun programme utilisateur ne peut définir de gestionnaire pour SIGKILL ou SIGSTOP, les rendant inévitables.

• Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus pour lui notifier un événement système ou lui ordonner une action spécifique.

• Gestionnaire de signal (Signal Handler) : Fonction logicielle personnalisée qu'un développeur peut attacher à un signal pour l'intercepter (catch) au lieu d'exécuter le comportement par défaut.

• SIGINT (Signal 2) : Émis lors du raccourci 'Ctrl + C' dans le terminal ; demande une interruption douce d'un processus interactif.

• SIGTERM (Signal 15) : Signal de terminaison propre envoyé par défaut par la commande 'kill ' ; autorise le processus à libérer ses ressources et sauvegarder ses données avant de s'éteindre.

• SIGKILL (Signal 9) : Ordre de destruction inconditionnel géré directement par le noyau Linux. STRICTEMENT IMPOSSIBLE à intercepter (catch), masquer ou ignorer par le programme.

• Job Control (SIGSTOP 19 / SIGCONT 18) : 'Ctrl + Z' émet le signal SIGSTOP pour suspendre l'exécution d'une tâche ; les commandes 'fg' (foreground) ou 'bg' (background) émettent SIGCONT pour reprendre l'exécution.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous disposez du temps nécessaire pour sauvegarder vos documents et fermer la porte à clé). SIGKILL (Signal 9) est une intervention d'urgence de la brigade de sécurité qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : Dans la syntaxe du shell Linux (Bash), le premier argument de la commande 'kill' désigne une option s'il commence par un tiret (ex: -9 ou -SIGKILL). Si l'on écrit 'kill 9 1234' sans tiret, le shell interprète '9' non pas comme un numéro de signal, mais comme un premier PID cible, et '1234' comme un second PID. Si le processus possédant le PID 9 n'existe pas ou appartient à un autre utilisateur, la commande échoue et renvoie une erreur d'accès ou d'inexistence.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus Zombie est un processus enfant qui a achevé son exécution mais dont le père n'a pas encore lu le code de retour (via l'appel système wait() ou waitpid()). Le processus Zombie ne consomme plus de mémoire RAM ni de CPU, mais conserve sa ligne d'immatriculation dans la table des processus du noyau. Il est STRICTEMENT IMPOSSIBLE de détruire un Zombie avec la commande 'kill -9 ' car le processus est DÉJÀ MORT ! Pour faire disparaître un Zombie, il faut forcer son processus père à lire son statut, ou détruire le processus père (kill -9). Le Zombie sera alors réadopté par systemd/init (PID 1) qui nettoiera immédiatement la table des processus.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant exécute la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le traitement réalisé lors de l'envoi du signal -15 et décrivez le résultat de la vérification ultérieure par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé à la fin de la première ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le shell affiche immédiatement l'identifiant de tâche (job ID, ex: [1]) ainsi que le PID attribué par le système (ex: 4521) et redonne instantanément la main à l'utilisateur pour lui permettre de taper de nouvelles commandes. Étape 2 : La commande 'ps -ef | grep sleep' interroge l'ensemble des processus du système d'exploitation et filtre les résultats pour afficher uniquement la ligne correspondant à notre commande 'sleep'. Cela permet de relever avec certitude son PID exact (4521) ainsi que son PPID (qui correspond au PID du shell Bash interactif). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus identifié. L'utilitaire 'sleep' ne disposant pas d'un gestionnaire personnalisé pour intercepter ce signal, il applique le comportement par défaut défini par le système d'exploitation : il libère ses ressources et s'arrête proprement.

Résultat : Le processus 'sleep 600' est définitivement arrêté de manière propre par le signal SIGTERM. Une nouvelle exécution de la commande 'ps -ef | grep sleep' confirme sa disparition totale de la table des processus actifs du système.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Il s'agit du processus 'systemd' (ou 'init' sur les anciennes distributions Unix/Linux), auquel le noyau attribue systématiquement le PID 1.

Q : Quel signal POSIX est transmis au processus s'exécutant au premier plan lorsqu'un utilisateur presse la combinaison de touches 'Ctrl + C' dans son terminal ?

R : C'est le signal SIGINT (Signal 2), qui demande l'interruption modérée d'une application interactive.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté, masqué ou ignoré par un programme informatique ?

R : Parce que selon la spécification POSIX, SIGKILL est pris en charge et exécuté directement par le noyau lui-même (kernel level). Le signal n'est jamais transmis au code de l'application utilisateur, empêchant tout gestionnaire de signal (signal handler) de s'interposer.